Univerzitet u Novom Sadu
FAKULTET TEHNIČKIH NAUKA

Departman za industrijsko inženjerstvo i inženjerski menadžment
Inženjerstvo informacionih sistema

PROJEKTNI ZADATAK IZ PREDMETA REINŽENJERING INFORMACIONIH SISTEMA

-SonarQube-

Andrijana Koljanin I720g2025

Novi Sad, 2026.

SADRŽAJ

Tekst zadatka	2
Opis korišćenih alata	2
Prikaz inicijalno detektovanih problema	8
Opis rešenja	15
Literatura	26

Tekst zadatka

U okviru projektog zadatka potrebno je izvršiti kompletnu analizu kôda korišćenjem platforme SonarQube. Analiza obuhvata projekte razvijane u okviru predmeta: Razvoj višeslojnih aplikacija (Spring i Angular) i Sistemi baza podataka (PL/SQL ili T-SQL). Student je dužan da izvrši sve korekcije koje SonarQube detektuje i da u konačnoj verziji kôda ne postoje upozorenja niti detektovani problemi. Ovaj dokument predstavlja detaljan opis rezultata analize i primenjenih rešenja.

Opis korišćenih alata

U okviru ovog projektnog zadatka izvršena je detaljna statička analiza izvornog koda korišćenjem platforme SonarQube, koja predstavlja jedan od najznačajnijih i najšire prihvaćenih alata za upravljanje kvalitetom softvera u savremenom razvoju informacionih sistema. SonarQube je namenjen kontinuiranom praćenju kvaliteta koda kroz identifikaciju potencijalnih grešaka, sigurnosnih rizika i loših programerskih praksi, sa ciljem unapređenja pouzdanosti, bezbednosti i dugoročne održivosti softverskih rešenja.

Primena SonarQube-a je široko rasprostranjena, kako u akademskom okruženju, gde se koristi kao edukativni alat za učenje principa kvalitetnog programiranja, tako i u industrijskim projektima, gde predstavlja sastavni deo procesa kontrole kvaliteta i kontinuirane integracije. Njegova vrednost ogleda se u mogućnosti da se problemi u kodu identifikuju u ranim fazama razvoja, čime se značajno smanjuju troškovi kasnijih korekcija i održavanja sistema.

Pored same SonarQube platforme, u procesu analize korišćen je i alat SonarScanner, koji predstavlja ključnu komponentu u procesu statičke analize. SonarScanner omogućava povezivanje izvornog koda sa SonarQube serverom i iniciranje procesa analize, pri čemu se izvorni kod ne izvršava, već se analizira na osnovu njegove strukture i sintakse. Ovakav pristup omogućava detekciju problema nezavisno od runtime okruženja i konkretnih ulaznih podataka U ovom poglavlju dat je detaljan i sveobuhvatan opis arhitekture SonarQube platforme, procesa statičke analize izvornog koda, klasifikacije detektovanih problema, kao i uloge SonarQube-a u unapređenju kvaliteta softverskih sistema.

Opis arhitekture SonarQube platforme

SonarQube platforma je temeljno arhitektonski koncipirana kao modularni, distribuirani sistem za kontinuiranu statičku analizu koda i upravljanje kvalitetom softvera, koji omogućava globalno usklađivanje metrike kvaliteta i sigurnosnih pravila kroz automatizovane procese u kontekstu DevOps i CI/CD okruženja. Ova arhitektura razdvaja analizu, obradu i skladištenje podataka kako bi se obezbedila skalabilnost, fleksibilnost i pouzdanost na nivou enterprise instalacija, i to kroz jasno diferencirane komponente koje međusobno interaguju, ali ostaju nezavisne u svojoj funkcionalnosti. Centralnu ulogu ima SonarQube Server, koji sadrži više ključnih procesa, uključujući Web Server za korisnički interfejs i REST API, Compute Engine za obradu izveštaja o analizi i indeksiranje podataka za brzo pretraživanje i prikaz metrika, dok Elasticsearch servis (koristi se za indeksiranje i optimizovano pretraživanje podataka) obezbeđuje performanse neophodne za navigaciju kroz kompleksne metrike i istorijske zapise kada je u pitanju velika količina koda. SonarQube Server radi u saradnji sa relacionom bazom podataka (npr. PostgreSQL, MS SQL ili Oracle), koja služi kao trajno skladište za konfiguracione podatke, profile pravila, metrike, istorijske rezultate analiza i zadatke za obradu, čime se omogućava odvojeno skaliranje obrade i trajnog skladištenja, što je posebno važno u okruženjima sa velikim brojem projekata i linija koda. Arhitektura je dodatno obogaćena plug-in ekosistemom, koji omogućava proširenje funkcionalnosti kroz jezičke specifične module, integracije

sa alatima trećih strana i dodatne rule-setove, što omogućava prilagođavanje platforme konkretnim potrebama subjektnih domena i razvojnih okruženja. Sa druge strane, SonarScanner klijenti, koji se pokreću lokalno ili kroz CI/CD pipeline (npr. integracijom sa Maven, Gradle, Jenkins ili GitHub Actions), ne obavljaju samu obradu, umesto toga oni generišu izlaz statičke analize i prenose ga serveru gde se vrši agregacija, izračunavanje metrika i klasifikacija problema po prioritetima i ovaj model omogućava da analiza koda bude ne samo integrisana sa procesom build-a već i da se ostvari kompatibilnost sa heterogenim tehnologijama i jezicima koristeći jedinstveni mehanizam metrika i pravila. SonarQube je identifikovan kao jedna od najraširenijih platformi za statičku analizu koda, koja omogućava merenje kvaliteta i detekciju problema kroz standardizovane metrike i pravila, dok empirijske studije pokazuju da je njegova arhitektura dovoljno fleksibilna da podrži različite jezike i paradigme, te da njegova upotreba dovodi do detekcije značajnog broja defekata u odnosu na druge static analysis alate (npr. SonarQube u poređenju sa PMD, Checkstyle i FindBugs) što potvrđuje njegove sposobnosti u krajnjem cilju poboljšanja kvaliteta softvera. Dalje, moderni evolucijski pravci SonarQube arhitekture ukazuju na uvođenje koncepta „architecture as code“, koji omogućava formalno deklarisanje arhitekture sistema u obliku YAML ili JSON konfiguracija i automatsku verifikaciju usklađenosti sa stvarnom implementacijom tokom CI/CD analiza. Ovakav arhitektonski dizajn, koji je distribuiran, skalabilan, moduliran i prilagodljiv, omogućava SonarQube platformi da se implementira kako u malim timovima, tako i u velikim korporativnim okruženjima sa zahtevima za visokom dostupnošću i obradom ogromnih baza koda, pri čemu server-klijent model i nezavisnost komponenti omogućavaju efikasnu automatizaciju i kontinualno upravljanje kvalitetom kroz integrisane metrike i pravila kvaliteta.

Opis rada SonarScanner-a i procesa analize

SonarScanner predstavlja ključnu komponentu SonarQube platforme i predstavlja specijalizovan klijent-alat za automatizovanu statičku analizu izvornog koda, čija je primarna funkcija da preuzme ulogu mehanizma koji pokreće i preprocesira statičku analizu u okviru CI/CD pipeline-a ili lokalnih build procesa, a zatim prenosi generisane rezultate serveru na dalju obradu i evaluaciju. Za razliku od tradicionalnih analiza koje se oslanjaju na izvršavanje programa u testnom okruženju, statička analiza koju SonarScanner olakšava zasniva se na analizi izvornog koda bez njegove izvršne faze, što omogućava rano otkrivanje potencijalnih grešaka, bezbednosnih ranjivosti i nepoželjnih obrazaca, istovremeno smanjujući troškove otklanjanja problema u kasnijim fazama softverskog životnog ciklusa. Proces rada SonarScanner-a započinje inicijalnom konfiguracijom projekta, gde se putem konfiguracionih fajlova ili integrisanih build sistema definiše opseg analize, uključujući određivanje putanja izvornog koda, selekciju jezika i primenu relevantnih pravila kvaliteta. Tokom pokretanja, SonarScanner prvo uspostavlja vezu sa SonarQube Server instancom kako bi preuzeo odgovarajuću verziju analitičkog engine-a i jezičkih analizatora, čime se obezbeđuje kompatibilnost alata i pravila sa specifičnim server-side konfiguracijama. Ovaj mehanizam dinamičkog preuzimanja komponenti omogućava da se alati za analizu ažuriraju centralno na serveru, a da klijentska okruženja ostanu jednostavna i usmerena na izvođenje analize, što predstavlja bitnu prednost u upravljanju statičkom analizom u velikim, heterogenim sistemima. Nakon preuzimanja neophodnih modula, SonarScanner prelazi u fazu skeniranja i parsiranja izvornog koda, gde se koriste sofisticirani analizatori za prepoznavanje abstract syntax tree (AST) struktura, metrika i obrazaca koji impliciraju potencijalne greške ili odstupanja od zadatih pravila. Ovaj korak obuhvata prepoznavanje sintaksičkih i semantičkih elemenata u kodu, identifikaciju struktura koje krše definisane standarde ili bezbednosne smernice, kao i izračunavanje ključnih metričkih pokazatelja kao što su kompleksnost, dupliranje koda, pokrivenost testovima i drugi atributi koji se upotrebljavaju za kvantitativnu ocenu kvaliteta. Napredne empirijske studije o statičkoj analizi ukazuju da alati poput SonarQube-a, koji kombinuju rule-based detekciju sa širokim spektrom metrika, značajno doprinose identifikaciji defekata i problema u kodu u poređenju sa tradicionalnim tehnikama, potvrđujući teorijsku osnovu za upotrebu

takvih pristupa u automatizovanim razvojnim okruženjima. Jedan od ključnih aspekata SonarScanner-ovog procesa je njegovo integrisano slanje struktuisanih izveštaja o analizi ka SonarQube Server-u nakon završetka skeniranja, gde se preko REST API-ja ili protokola za komunikaciju predaju sirovi podaci o problemima, metrikama i detekcijama. Na serveru se ti podaci obrađuju asinhrono, odnosno metrika se agregira, pravila evaluiraju, a rezultati se upoređuju sa unapred definisanim quality gates kriterijumima koji služe kao pragovi kvaliteta za prihvatanje ili odbacivanje određenih promena u kodnoj bazi. Ovaj model omogućava integraciju statičke analize direktno u CI/CD tokove, tako da se promene u kodu automatski analiziraju pri svakom commit-u ili pull request-u, pružajući timu povratnu informaciju o kvalitetu u realnom vremenu i omogućavajući shift-left strategiju ranog otkrivanja problema, čime se smanjuje rizik od skupljih popravki u kasnijim fazama razvoja ili produkcionom okruženju. U konačnom ishodu, SonarScanner predstavlja efikasan most između lokalnih razvojnih procesa i centralizovanog sistema evaluacije kvaliteta koda, koji pruža doslednu, automatizovanu i merljivu osnovu za unapređenje kvaliteta softvera i usklađivanje sa definisanim standardima kvaliteta.

Vrste detektovanih problema i način klasifikacije

SonarQube primenjuje sofisticiranu klasifikaciju detektovanih problema (issues) kao centralni element svog modela kvaliteta koda, oslanjajući se na princip statičke analize koji omogućava identifikovanje različitih devijacija od standarda kvaliteta bez izvršavanja programa. U osnovi ove klasifikacije nalazi se struktura problema koja obuhvata Bugs, Vulnerabilities i Code Smells, dok se ponekad vodi računa i o Security Hotspots, kao zasebnoj kategoriji koja ukazuje na sigurnosno relevantne konstrukcije koje zahtevaju ručnu procenu, ali ne predstavljaju automatski potvrđene ranjivosti.

Bugs su definisani kao greške u logici ili strukturi koda koje verovatno dovode do pogrešnog ponašanja ili pada sistema tokom izvršavanja, implicirajući narušavanje pouzdanosti softvera i potencijalnu funkcionalnu neispravnosti. Ovakve greške obuhvataju, na primer, dereferenciranje null pokazivača, neadekvatno upravljanje izuzecima i one su klasifikovane kao najhitnije za otklanjanje zbog njihove sposobnosti da uzrokuju kvarove.

Vulnerabilities predstavljaju sigurnosne slabosti u logici aplikacije ili njenim zavisnostima koje mogu omogućiti izvršenje napada, kao što su SQL injekcije, osetljivi podaci hard-kodirani u izvornom kodu, ili propusti u validaciji ulaza, a njihova detekcija je kritična za prevenciju potencijalnih bezbednosnih eksploatacija u produkcionim okruženjima.

Code Smells označavaju obrasce kodnih struktura koje funkcionalno ne utiču direktno na ponašanje sistema, ali ukazuju na loše prakse dizajna i održivosti koje vremenom povećavaju tehnički dug, smanjuju čitljivost i otežavaju modifikacije. Primeri obuhvataju prevelike metode, duplirane kodne blokove, nejasne nazive ili neefikasnu organizaciju paketa. Pored ovih primarnih tipova, u nekim SonarQube konfiguracijama javlja se i kategorija Security Hotspot, koja sadrži delove koda koji su potencijalno sigurnosno osetljivi ali zahtevaju ljudski pregled kako bi se odredilo da li su zaista ranjivi, čime se podržava balans između automatizovane identifikacije i stručne procene u domenu bezbednosti. Klasifikacija problema je dodatno obogaćena nivoima ozbiljnosti (severity) kao što su Blocker, Critical, Major, Minor i Info, što omogućava kvantitativnu i prioritetnu razliku među detektovanim problemima, tako da timovi mogu fokusirati napore na one koji imaju najveći uticaj na pouzdanost, sigurnost ili održivost sistema. U empirijskim studijama o SonarQube upotrebi jasno se ukazuje da ova višestruka klasifikacija ne samo da strukturira povratne informacije o kvalitetu koda, već omogućava i napredne evaluacije tehničkog duga putem agregacije metrika, pri čemu se kodne slabosti i obrasci održavanja kvantifikuju u merama koje se mogu koristiti za strateško planiranje refaktorisanja. U nekim slučajevima, kao što su opsežne analize velikih open-source projekata,

kontinuirano praćenje ovih klasifikovanih problema služi kao indikator trendova promena kvaliteta tokom vremena i omogućava kvantitativno poređenje različitih komponenata sistema. Ovakav pristup strukturiranog kategorizovanja problema omogućava organizacijama da ne samo detektuju probleme u ranoj fazi razvoja, već i da ih prioritetno adresiraju u skladu sa njihovom potencijalnom štetom, čime se unapređuje opšti kvalitet, sigurnost i održivost softverskih proizvoda u kompleksnim razvojnim okruženjima.

Prikaz interfejsa, dashboard-a i metrika

SonarQube-ov korisnički interfejs i dashboard predstavljaju centralnu komponentu vizualizacije kvaliteta softvera, kroz koju razvojni timovi dobijaju kvantitativnu i intuitivnu povratnu informaciju o rezultatu statičke analize koda. Dashboard omogućava pregled ključnih metrika na jednom mestu, osiguravajući uvid u stanje kvaliteta, trendove promena kroz vreme i status definisanih uslova kvaliteta koji se koriste u procesu odlučivanja o sledećim koracima u razvoju. Jedna od najvažnijih funkcionalnosti je prikaz Quality Gate statusa, koji služi kao objektivni indikator da li kodna baza zadovoljava unapred definisane pragove kvaliteta. Ovaj status se obično prikazuje kao jasno označena vrednost (npr. OK ili ERROR), čime se omogućava brzo vizuelno procenjivanje da li kod može biti dalje integrisan ili pušten u produkciju. Quality Gate se formira kombinovanjem više uslova baziranih na metrici kao što su pokrivenost testovima, duplikacije koda, broj novih grešaka, kao i ocene pouzdanosti, sigurnosti i održivosti, i ukoliko bilo koji od ovih uslova ne zadovolji zadate pragove, Quality Gate automatski označava da projekat nije uspeo proveru kvaliteta. Ova automatska evaluacija služi kao integrisana kontrola kvaliteta unutar CI/CD pipeline-a i omogućava razvojnom timu da predupredi degradaciju kvaliteta.

Na dashboard-u se vizualizuju ocene pouzdanosti (Reliability Rating), koje se baziraju na broju i ozbiljnosti reliability issues, grešaka koje mogu dovesti do runtime problema ili nepredviđenog ponašanja sistema. Ove ocene su često predstavljene kroz skalu (npr. A do E) koja omogućava brzo neusaglašavanje sa standardima pouzdanog softvera. Slično tome, sigurnosne metrike (Security) obuhvataju broj detektovanih ranjivosti i njihov nivo ozbiljnosti, kao i ocenu security rating, čime se omogućava timovima da kvantifikuju izloženost potencijalnim napadima ili sigurnosnim propustima pre nego što kod dospe u krajnje okruženje. Ove metrike se posebno ističu na dashboard-u jer je sigurnost danas integralan deo kvaliteta softvera i koriste se i za upravljanje rizikom i za usklađivanje sa standardima bezbednosti.

Metrike održivosti (Maintainability) predstavljaju još jedan ključni segment vizualizacije, obuhvatajući identifikaciju code smells, tehničkog duga i procene napora potrebnog za refaktorisanje. Ove metrike kvantifikuju koliko je teško održavati i unapređivati kodnu bazu tokom vremena. Dashboard ih obično prikazuje kao ukupne vrednosti i kao trendove koji omogućavaju sagledavanje promene održivosti kroz serije build-ova. Pored ovih glavnih kategorija, SonarQube takođe prikazuje dodatne metrike kao što su pokrivenost koda testovima (code coverage), duplirani kod (duplicated code) i kompleksnost koda (cyclomatic i cognitive complexity), koje su vizualno predstavljene kroz grafikone i indikatore koji pomažu timovima da identifikuju delove koda koji zahtevaju dodatnu pažnju ili refaktorisanje, te da bolje shvate strukturu i rizike u softverskom sistemu.

Dashboard takođe omogućava filtriranje i segmentaciju metrika na nivou celog projekta ili samo za novi kod, što omogućava ciljanu evaluaciju poslednjih promena bez uticaja istorijskih problema, što je posebno korisno u agilnim timovima koji rade iterativno. Kroz interaktivne prikaze i detaljne listinge relevantnih problema, korisnik može brzo identifikovati konkretne instance koje doprinose lošim ocenjivanjima metrika i dobiti preporuke za poboljšanje ili dalju akciju. Ovakav detaljan i empirijski utemeljen prikaz metrika na dashboard-u omogućava ne samo povratne informacije o trenutnom stanju, već i praćenje evolucije kvaliteta tokom vremena, te podržava donošenje odluka zasnovanih na

podacima u profesionalnom okruženju razvojnih timova, čime SonarQube predstavlja snažan alat za sistematsko upravljanje kvalitetom koda u savremenim softverskim procesima.

Integracija sa projektima i razvojnim alatima

Ključni koncept koji omogućava široku primenu SonarQube-a jeste njegova integracija sa razvojnim alatima i CI/CD ekosistemima, što omogućava da analiza kvaliteta koda ne bude izolovana nakon implementacije, već integrisana u standardne tokove rada kroz automatizovane build i deploy procese. Konkretno, korišćenjem SonarScanner-a u kombinaciji sa alatima za automatizaciju build procesa kao što su Maven i Gradle postiže se da se analiza pokreće automatski pri svakom sastavljanju koda, što eliminiše potrebu za manuelnim pokretanjem analiza i uveliko smanjuje latentni tehnički dug time što se problemi identifikuju u trenutku njihovog pojavljivanja, pre nego što eskaliraju u ozbiljnije defekte u proizvodnom okruženju. SonarQube se integriše sa vodećim DevOps alatima za kontinuiranu integraciju I isporučivanje, uključujući Jenkins, GitLab CI/CD, GitHub Actions i Azure DevOps, omogućavajući programerima i timovima da definišu tzv. quality gate kriterijume - formalizovane pragove kvaliteta kojima kod mora da zadovolji pre nego što se automatski izvrši njegovo spajanje u glavni branch ili pre nego što se pokrene produkcioni deploy. Takav pristup ne samo da unapređuje tehničku disciplinu unutar tima, već služi i kao objektivan mehanizam za smanjenje brojnih klasičnih softverskih rizika u velikim i kompleksnim projektima. Pored podrške za tradicionalne backend jezike, integracija SonarQube-a obuhvata i alate koji se koriste u modernim full-stack i frontend okruženjima, kao što su Node.js, JavaScript i TypeScript, čime se omogućava dosledna analiza kvaliteta i na nivou korisničkog interfejsa i poslovne logike. Ova interoperabilnost je od značaja u projektima koji koriste tehnologije i arhitekture zasnovane na mikroservisima i komponentama, jer omogućava objedinjeni uvid u kvalitet svih segmenta sistema. Integracija sa bazama podataka i skriptnim jezicima, uključujući PL/SQL i T-SQL, omogućava da se kvalitet ne meri samo na nivou aplikacijskog koda već i u najkritičnijim tačkama poslovne logike koja se izvršava u sloju podataka, gde su performanse i sigurnost od ključnog značaja. Pored toga, SonarQube podržava i proširenja kroz plug-in arhitekturu i SonarLint integraciju u IDE-je, što omogućava ranu detekciju problema direktno u razvojnom okruženju pre nego što promene napuste lokalnu mašinu programera. Ovakav pristup, u kojem se statička analiza i integracija sa alatima za kontinuiranu isporuku uspostavi kao standardni deo radnog procesa, ne samo da smanjuje učestalost regresija u kodu već i unapređuje saradnju unutar timova kroz zajedničko razumevanje metrika kvaliteta i ciljeva koje je potrebno postići. Sve ove karakteristike čine SonarQube ne samo tehničkim alatom za procenu koda, već i strateškim resursom za upravljanje kvalitetom u složenim softverskim projektima, koji podržava principe DevOps kulture, ranog otkrivanja defekata i kontinuiranog poboljšanja kroz automatizovane povratne informacije i integrisane metrike kvaliteta.

Podešavanje pravila i profila kvaliteta

Podešavanje pravila i profila kvaliteta predstavlja jedan od ključnih mehanizama SonarQube platforme koji omogućava prilagođavanje procesa statičke analize specifičnim zahtevima projekta, organizacije ili domena primene. Profil kvaliteta (Quality Profile) u SonarQube-u definiše skup pravila koja se primenjuju nad izvornim kodom za određeni programski jezik i predstavlja formalizovanu interpretaciju standarda dobrih praksi u razvoju softvera. Svaki projekat u okviru SonarQube sistema je povezan sa jednim aktivnim profilom kvaliteta, čime se obezbeđuje konzistentna i ponovljiva analiza koda kroz sve iteracije razvoja. Pravila unutar profila obuhvataju različite aspekte kvaliteta, uključujući pouzdanost, sigurnost i održivost koda, a njihova primena omogućava identifikaciju potencijalnih grešaka, bezbednosnih ranjivosti i loših programerskih praksi pre izvršavanja aplikacije. Posebna prednost ovog pristupa ogleda se u mogućnosti prilagođavanja profila specifičnim ciljevima projekta, kao što su stroži bezbednosni zahtevi u sistemima koji obrađuju osetljive podatke ili povećan fokus na održivost i čitljivost koda u dugoročno održavanim

informacionim sistemima. Svako pravilo u okviru profila poseduje definisan nivo ozbiljnosti, koji se koristi za klasifikaciju detektovanih problema na minor, major, critical i blocker, čime se omogućava efikasna prioritizacija korektivnih aktivnosti i racionalno upravljanje tehničkim dugom. Ovakva hijerarhijska klasifikacija problema podržava donošenje odluka zasnovanih na riziku, što je posebno važno u industrijskim okruženjima sa ograničenim resursima i kratkim rokovima isporuke. Dodatno, SonarQube omogućava organizacijama da kreiraju i održavaju sopstvene profile kvaliteta, čime se postiže standardizacija razvoja softvera na nivou tima ili preduzeća, dok se istovremeno zadržava fleksibilnost za prilagođavanje različitim tipovima projekata. U kontekstu reinženjeringa informacionih sistema, profili kvaliteta imaju značajnu ulogu jer omogućavaju postepeno pooštravanje kriterijuma kvaliteta, čime se nasleđeni sistemi mogu unapređivati iterativno, bez narušavanja stabilnosti i funkcionalnosti postojećih rešenja. Na taj način, SonarQube profili kvaliteta predstavljaju efikasan alat za upravljanje kvalitetom softvera u celokupnom životnom ciklusu sistema, od inicijalnog razvoja do dugoročnog održavanja i modernizacije.

Prednosti korišćenja SonarQube-a u industrijskim projektima

SonarQube se u industrijskim softverskim projektima pokazao kao jedna od najefikasnijih platformi za automatizovanu statičku analizu koda i upravljanje kvalitetom kodne baze, jer omogućava organizacijama da unaprede pouzdanost, sigurnost i održivost softverskih proizvoda kroz doslednu primenu metrike kvaliteta i quality gate pravila koja se integrišu direktno u razvojne tokove. Jedna od osnovnih prednosti njegove upotrebe jeste sposobnost ranog otkrivanja grešaka i ranjivosti, čime se smanjuje rizik od kasnih defekata u produkciji i značajno smanjuju troškovi popravki u kasnijim fazama životnog ciklusa softvera, jer problemi koji se identifikuju u ranoj fazi razvoja zahtevaju mnogo manje resursa za otklanjanje. Osim toga, SonarQube omogućava organizacijama da standardizuju i automatski sprovode politike kvaliteta koda kroz Quality Gate mehanizme, koji služe kao pragovi kvaliteta u CI/CD pipeline-ima i sprečavaju integraciju i distribuciju koda koji ne zadovoljava definisane kriterijume, što doprinosi stabilnosti i pouzdanosti velikih sistemskih rešenja. U industrijskoj praksi, ova kontrola kvaliteta ne samo da povećava povjerenje u stabilnost softvera već i olakšava usklađivanje sa zahtevima regulatornih standarda i bezbednosnih okvira, što je naročito značajno u domenima kao što su finansijske usluge, zdravstvo i telekomunikacije gde su sigurnost i usklađenost sa standardima ključni. Pored toga, SonarQube omogućava evaluaciju tehničkog duga, dajući timovima metrike koje kvantifikuju strukturalne probleme poput dupliranog koda, visoke kompleksnosti i neefikasnih obrazaca, čime se poboljšava održivost i olakšava planiranje refaktorisanja u dugoročnim projektima. Integracija sa brojnim programskim jezicima i razvojim alatima omogućava industrijskim timovima da primene konzistentne prakse kvaliteta bez obzira na tehnološki stack, a podrška za više od 35 jezika omogućava uniformnu procenu kvaliteta u heterogenim okruženjima. Takođe, vizualizacija metrika i dashboard-i koje SonarQube pruža daju jasnu i merljivu povratnu informaciju o stanju kvaliteta, uključujući ocene pouzdanosti, sigurnosti, održivosti i pokrivenosti testovima, što olakšava menadžmentu i timovima donošenje strateških odluka zasnovanih na empirijskim podacima. Na nivou tima, SonarQube povećava produktivnost tako što smanjuje dupliranje truda i omogućava developerima da se fokusiraju na unapređenje funkcionalnosti umesto saniranja pogrešaka kasnije u procesu, a real-time povratne informacije i integracije sa IDE-jevima dodatno ubrzavaju učenje i primenu dobrih programerskih praksi. Sve ove prednosti, od ranog otkrivanja defekata, standardizovane primene pravila kvaliteta, praćenja tehničkog duga, do poboljšanja produktivnosti i usklađenosti sa industrijskim standardima, čine SonarQube kritičnim alatom za industrijske projekte koji teže visokom nivou kvaliteta, pouzdanosti i sigurnosti softverskih proizvoda.

Prikaz inicijalno detektovanih problema

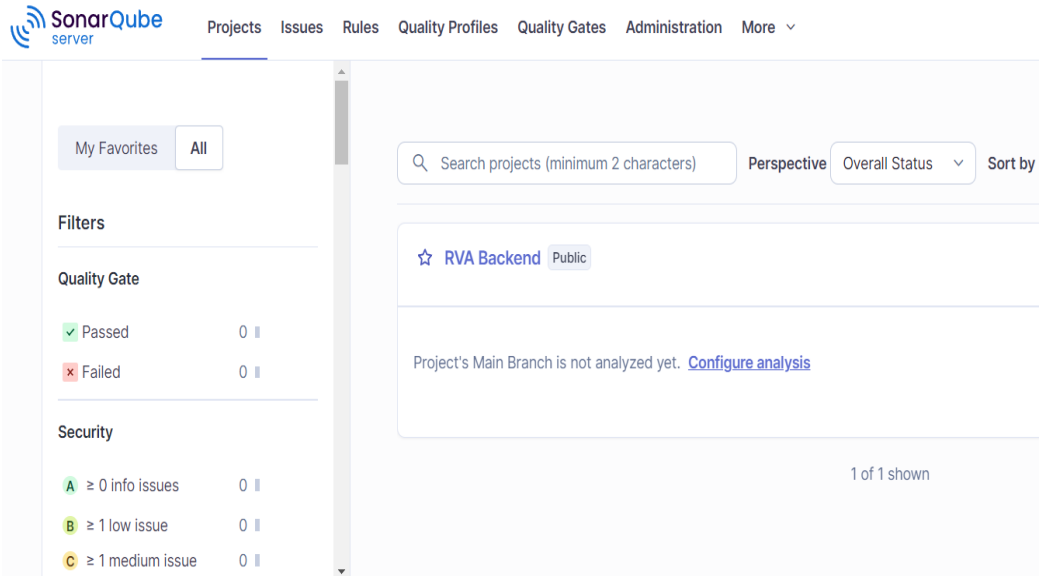
Tokom inicijalne analize SonarQube je detektovao više kategorija problema, uključujući:
- Bug-ove (logičke greške u kôdu)
- Sigurnosne ranjivosti
- Code smell-ove povezane sa dupliranim kôdom, složenošću i lošim praksama
- Problemi sa formatiranjem i stilom kôda
- Problemi specifični za SQL skripte (neoptimalni upiti, deprecated sintaksa i sl.)

U nastavku dokumenta detaljno su opisane sve kategorije problema uz primere i tumačenja.

Maven projekat

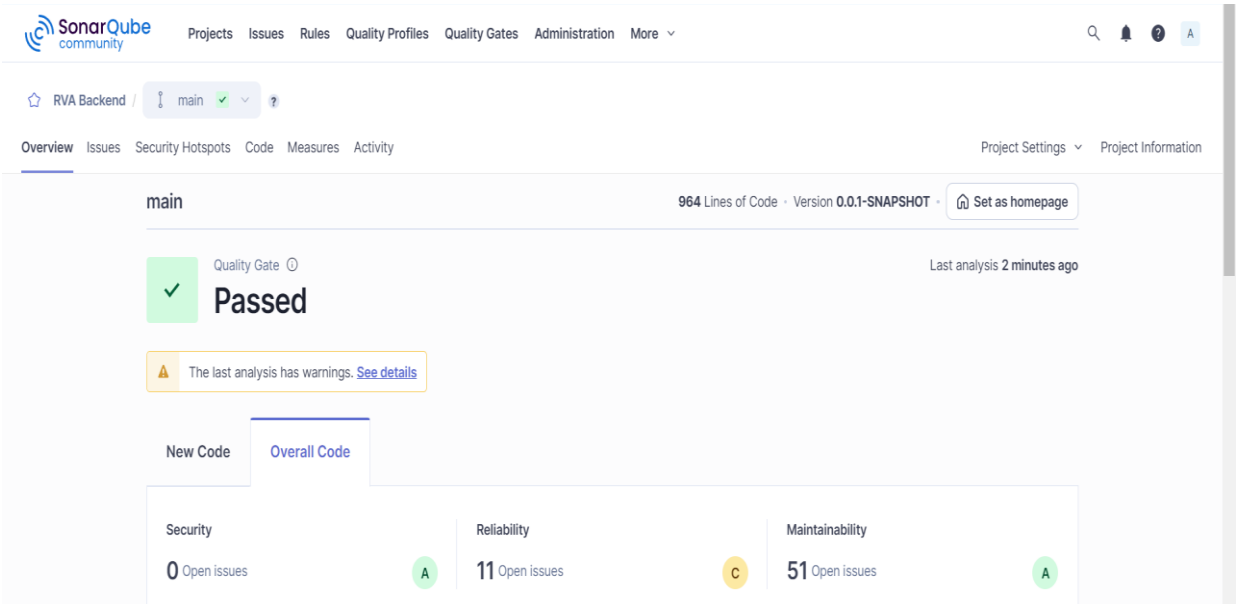
Tokom inicijalne statičke analize backend Maven projekta, sprovedene primenom alata SonarQube, izvršena je sistematska procena kvaliteta izvornog koda sa ciljem identifikovanja potencijalnih grešaka, loših programerskih praksi i elemenata koji mogu negativno uticati na dugoročnu održivost softverskog sistema. Analiza je obuhvatila kompletan serverski deo aplikacije, uključujući REST kontrolere, servisni sloj, domenski model i integracione testove.

Na slici ispod je prikazan SonarQube alat sa uveženim Spring delom projekta (RVA Backend), pri čemu analiza glavne grane još uvek nije pokrenuta.



Slika 1: Uvoženje Spring dela projekta u SonarQube alat

Nakon uspešnog uvoženja projekta, pokrenuta je analiza glavne grane, čiji su rezultati prikazani na slici ispod. Quality Gate status je označen kao Passed, sa ukupno 964 linije koda, bez bezbednosnih propusta, ali uz 11 problema pouzdanosti i 51 problem održivosti koda.



Slika 2: Prikaz inicijalne analize backend Maven projekta

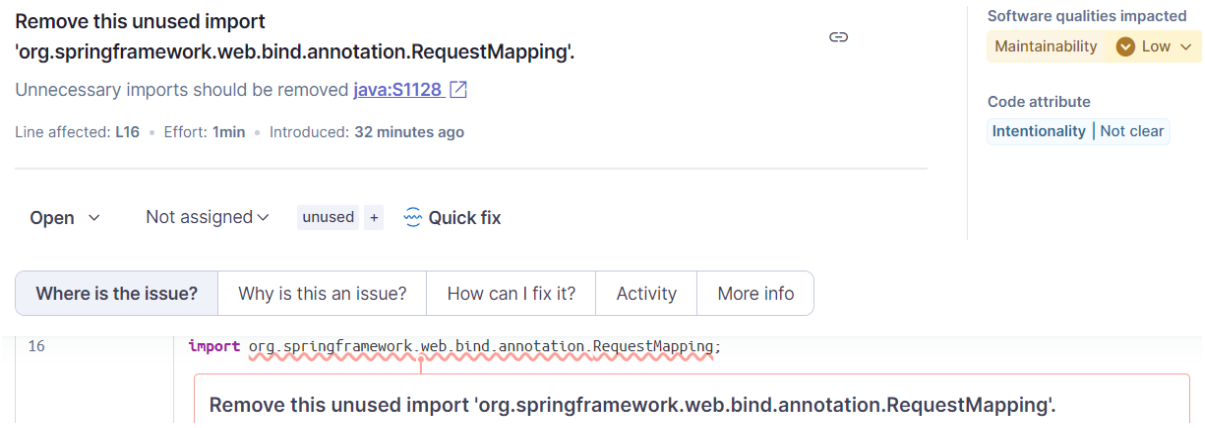
Rezultati analize pokazali su da projekat ne sadrži kritične bezbednosne propuste niti ozbiljne funkcionalne greške koje bi direktno ugrozile izvršavanje aplikacije, što potvrđuje i pozitivan Quality Gate status. Međutim, identifikovan je značajan broj problema koji se odnose na stil pisanja koda, doslednost, strukturu i primenu preporučenih dobrih praksi u okviru Spring i Java ekosistema, pre svega u kategoriji održivosti, sa 51 registrovanim problemom, kao i 11 problema u kategoriji pouzdanosti. Ovi problemi, iako pojedinačno ne predstavljaju neposrednu pretnju stabilnosti sistema, mogu kumulativno dovesti do povećanja tehničkog duga i otežanog održavanja aplikacije u budućnosti, te su kao takvi uzeti u razmatranje u okviru procesa refaktorisanja.

Detektovane kategorije problema:

- **Neiskorišćeni importi**

Jedna od dominantnih kategorija identifikovanih problema odnosi se na prisustvo neiskorišćenih importa u velikom broju klasa. Ovakve nepravilnosti registrovane su u REST kontrolerima, servisnim interfejsima, servisnim implementacijama, model klasama.

Na slici ispod je prikazan primer problema neiskorišćenog importa detektovanog od strane SonarQube alata. Konkretno, uvezena klasa `org.springframework.web.bind.annotation.RequestMapping` nije upotrebljena u kodu, a njen uticaj klasifikovan je kao nizak po pitanju održivosti, uz preporučenu akciju uklanjanja putem opcije Quick fix.



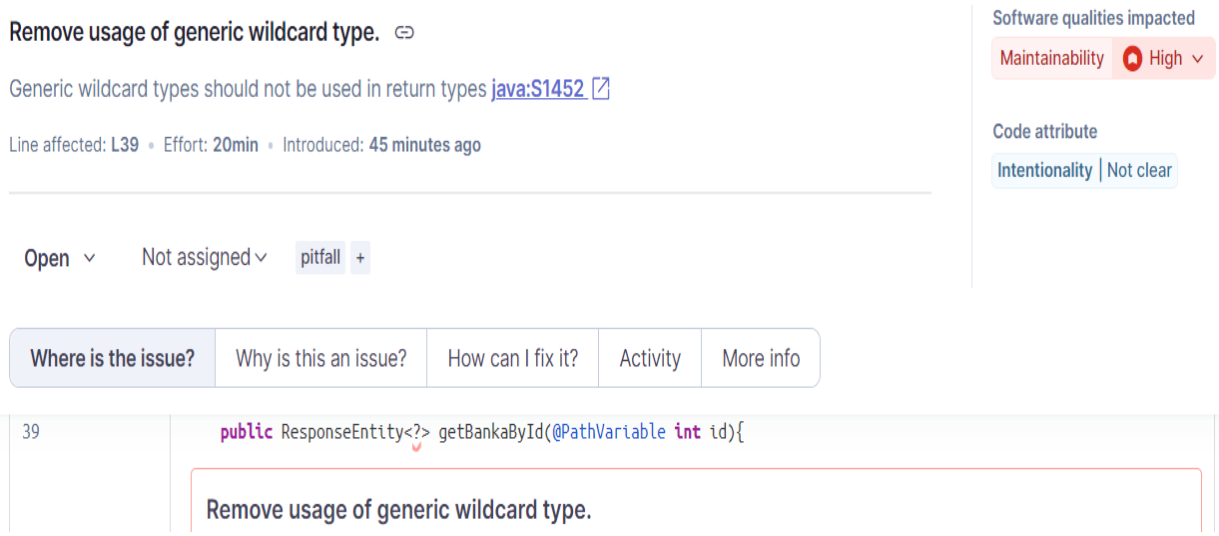
Slika 3: Prikaz problema neiskorišćenih importa

Neiskorišćeni importi ne utiču direktno na izvršavanje programa, ali značajno narušavaju čitljivost i preglednost koda. Njihovo prisustvo može dovesti do pogrešne interpretacije funkcionalnosti klase, jer sugeriše upotrebu određenih tipova ili zavisnosti koje zapravo nisu deo implementacije. Sa aspekta održivosti, ovakva praksa otežava refaktorisanje i povećava kognitivno opterećenje programera koji se naknadno upoznaje sa kodom. Uklanjanjem neiskorišćenih import izjava postiže se jasnija struktura koda i veći stepen profesionalne urednosti.

Korišćenje generičkog wildcard tipa

Analiza je otkrila veliki broj instanci korišćenja generičkog wildcard tipa, posebno u povratnim vrednostima metoda REST kontrolera, gde se kao povratni tip koristi ResponseEntity<?>. SonarQube ovakvu praksu klasifikuje kao problem visokog prioriteta, jer direktno utiče na jasnoću interfejsa i sigurnost aplikacije.

Na slici ispod je prikazan problem visokog prioriteta koji se odnosi na upotrebu generičkog wildcard tipa <?> kao povratne vrednosti metode `getBankaById` u REST kontroleru. Ovakva praksa direktno narušava jasnoću interfejsa i sigurnost tipova, a SonarQube je klasifikuje kao problem visoke važnosti po pitanju održivosti koda.



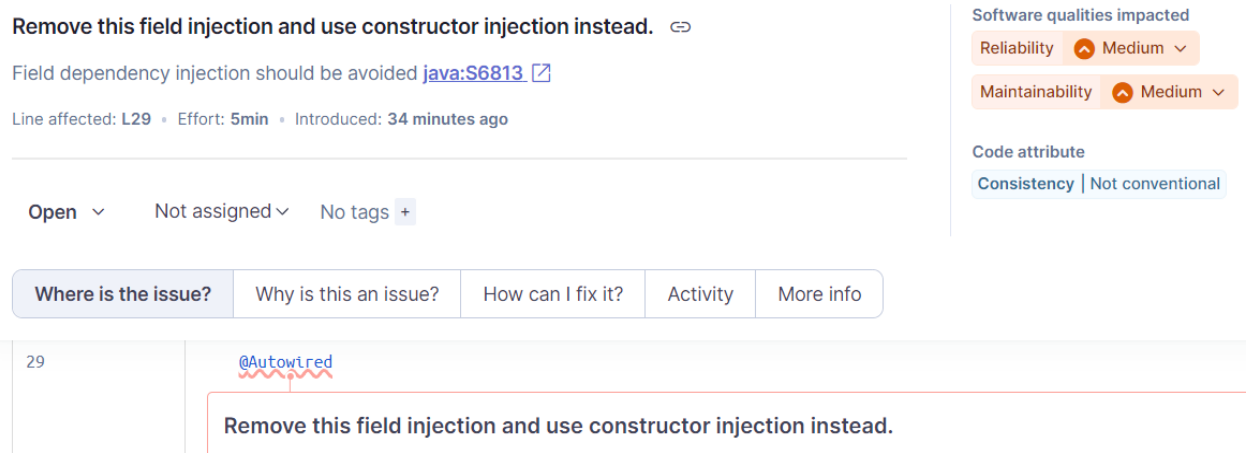
Slika 4: Prikaz problema generičkog wildcard tipa

Upotreba generičkog wildcard tipa prikriva stvarni tip podataka koji metoda vraća, čime se narušava eksplicitnost ugovora između serverske i klijentske strane. Ovakav pristup otežava statičku analizu koda, smanjuje mogućnost pravovremenog otkrivanja grešaka tokom razvoja i može dovesti do nejasnoća prilikom daljeg proširenja sistema. Preporučuje se definisanje konkretnih generičkih tipova, čime se unapređuje razumljivost API-ja, olakšava održavanje i poboljšava kvalitet dokumentacije.

- **Field injection umesto constructor injection**

U servisnom i kontrolerskom sloju aplikacije uočen je obrazac korišćenja field injection mehanizma za ubrizgavanje zavisnosti, najčešće putem anotacije `@Autowired`. SonarQube ovaj pristup označava kao nepovoljan i preporučuje korišćenje `constructor injection` tehnike.

Na slici ispod je prikazano upozorenje srednje važnosti koje ukazuje na korišćenje `field injection` pristupa putem `@Autowired` anotacije. Problem je klasifikovan kao relevantan i za pouzdanost i za održivost koda, jer narušava konzistentnost i uobičajene konvencije Spring ekosistema.



Slika 5: Prikaz problema field injection-a detektovanog u SonarQube alatu

Field injection smanjuje transparentnost zavisnosti jedne klase, jer one nisu jasno vidljive u njenom konstruktoru. Takođe, ovakav pristup otežava pisanje unit testova, budući da zahteva korišćenje refleksije ili dodatnih testnih konfiguracija. Constructor injection, sa druge strane, omogućava eksplicitno definisanje svih zavisnosti, povećava testabilnost i olakšava kontrolu nad životnim ciklusom objekata, čime se unapređuje pouzdanost i stabilnost sistema.

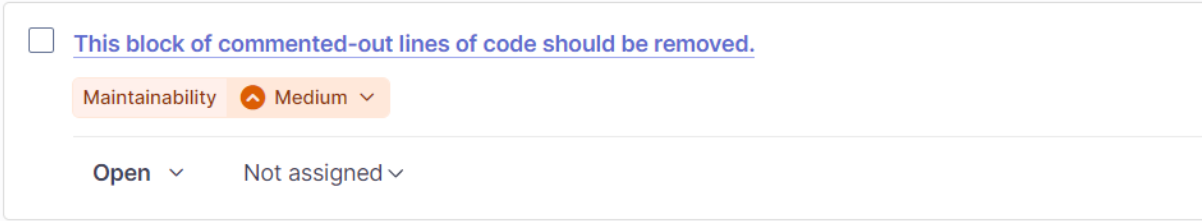
- **Dupliranje literala i dizajnerske nepravilnosti**

Tokom analize identifikovani su slučajevi ponavljanja identičnih string literala u više metoda, naročito poruka koje obaveštavaju korisnika da traženi resurs ne postoji. SonarQube ovakvu praksu klasifikuje kao dizajnersku nepravilnost visokog prioriteta.

Dupliranje literala predstavlja kršenje principa jedinstvene odgovornosti i otežava održavanje sistema. U slučaju izmene poruke, neophodno je menjati kod na više mesta, što povećava rizik od grešaka i nekonzistentnosti. Centralizacijom ovakvih vrednosti kroz definisanje konstanti ili korišćenjem zajedničkih klasa za poruke postiže se veća fleksibilnost, konzistentnost i dugoročna održivost aplikacije.

Neiskorišćeni i komentarisani delovi koda u testovima

U okviru integracionih testova identifikovani su blokovi zakomentarisnog koda koji više nemaju funkcionalnu svrhu. Ovakvi delovi koda predstavljaju oblik tehničkog duga i mogu dovesti do konfuzije prilikom analize testnog sloja aplikacije.



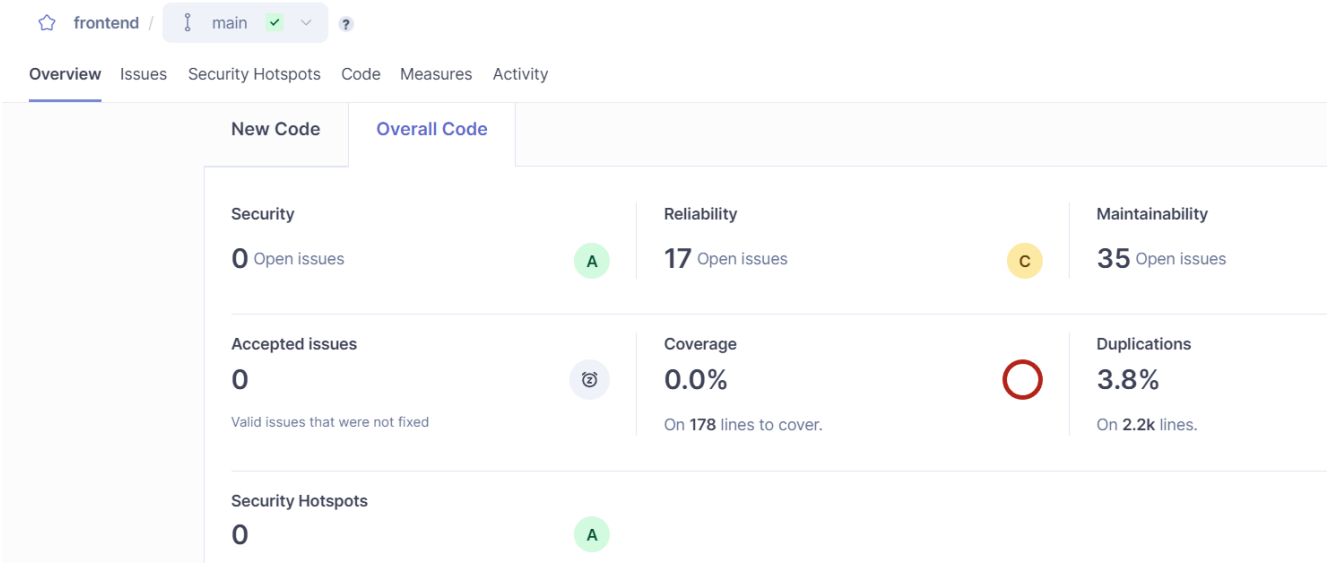
Slika 6: Prikaz problema komentarisanih delova koda

Prisustvo zastarelog i neiskorišćenog koda smanjuje čitljivost testova, otežava razumevanje testnih scenarija i može negativno uticati na poverenje u validnost testnog skupa. Uklanjanjem ovih delova koda postiže se jasnija struktura testova, veća preglednost i lakše održavanje testnog okruženja.

Angular projekat

Analiza rezultata statičke analize koda pomoću SonarQube alata.

Tokom inicijalne statičke analize frontend projekta, sprovedene primenom SonarQube Scanner alata, izvršena je detaljna procena kvaliteta klijentskog dela aplikacije razvijenog korišćenjem Angular framework-a. Analiza je obuhvatila kompletan frontend sloj sistema, uključujući komponente, dijaloge, servise, HTML templejte, CSS stilove, kao i osnovnu konfiguraciju aplikacije.



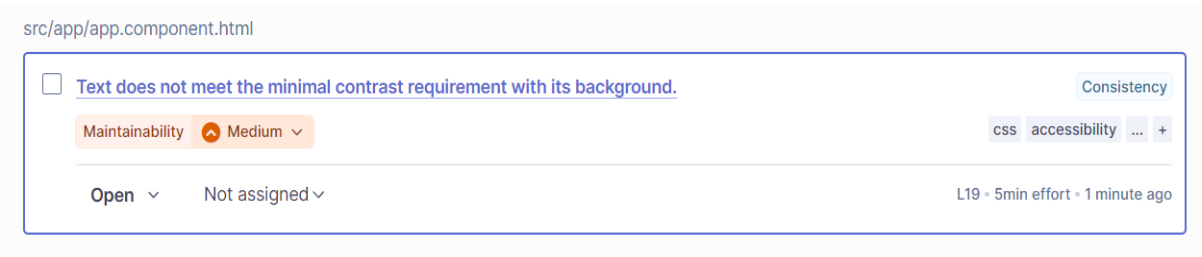
Slika 7: Prikaz inicijalne analize Angular projekta

Rezultati analize pokazali su da frontend projekat ne sadrži kritične bezbednosne ranjivosti niti funkcionalne greške koje bi direktno ugrozile izvršavanje aplikacije u produkcionom okruženju. Međutim, identifikovan je značajan broj problema koji se odnose na stil pisanja koda, pravilnu upotrebu TypeScript sintakse, doslednost u implementaciji Angular obrazaca, kao i na pristupačnost korisničkog interfejsa. Iako ovi problemi pojedinačno ne dovode do neispravnog rada aplikacije, njihovo kumulativno prisustvo može negativno uticati na čitljivost koda, održavanje sistema i korisničko iskustvo.

Detektovane kategorije problema:

- **Problemi pristupačnosti i kontrasta teksta**

Jedna od identifikovanih kategorija problema odnosi se na neadekvatan kontrast između boje teksta i pozadine u HTML i CSS fajlovima aplikacije. SonarQube je registrovao da pojedini tekstualni elementi ne zadovoljavaju minimalne zahteve kontrasta definisane smernicama za pristupačnost (WCAG standard).

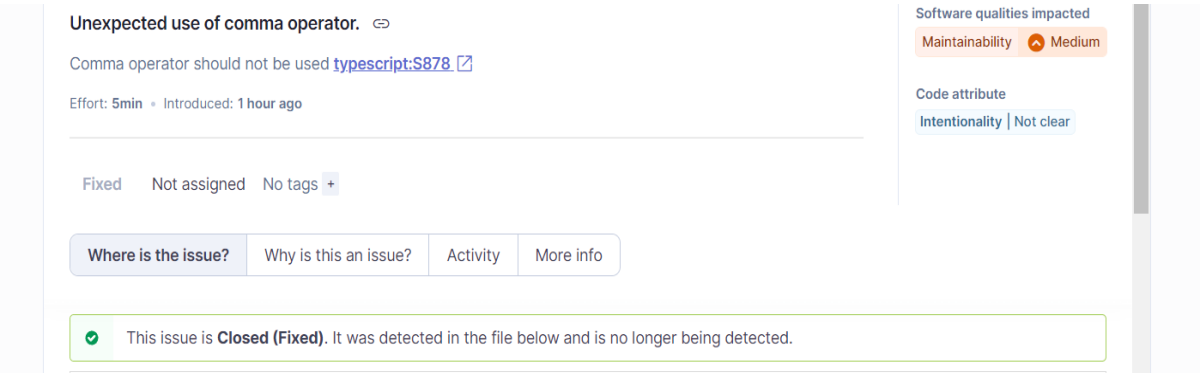


Slika 8: Prikaz problema kontrasta teksta

Ovakvi problemi ne utiču direktno na funkcionalnost aplikacije, ali mogu značajno otežati korišćenje sistema osobama sa smanjenim vidom, kao i u uslovima slabije vidljivosti. Povećanjem kontrasta između teksta i pozadine, kroz prilagođavanje CSS stilova i boja, unapređena je pristupačnost aplikacije i usklađenost sa savremenim standardima korisničkog interfejsa.

- **Nepravilna upotreba zareznog (comma) operatora**

Analiza je otkrila više instanci nepravilne upotrebe zareznog operatora u TypeScript fajlovima, naročito unutar metoda koje obrađuju asinhrone operacije putem Observable objekata. Ovakva sintaksa dovodi do toga da se izrazi tumače kao neiskorišćene ekspresije, što SonarQube klasifikuje kao problem pouzdanosti.

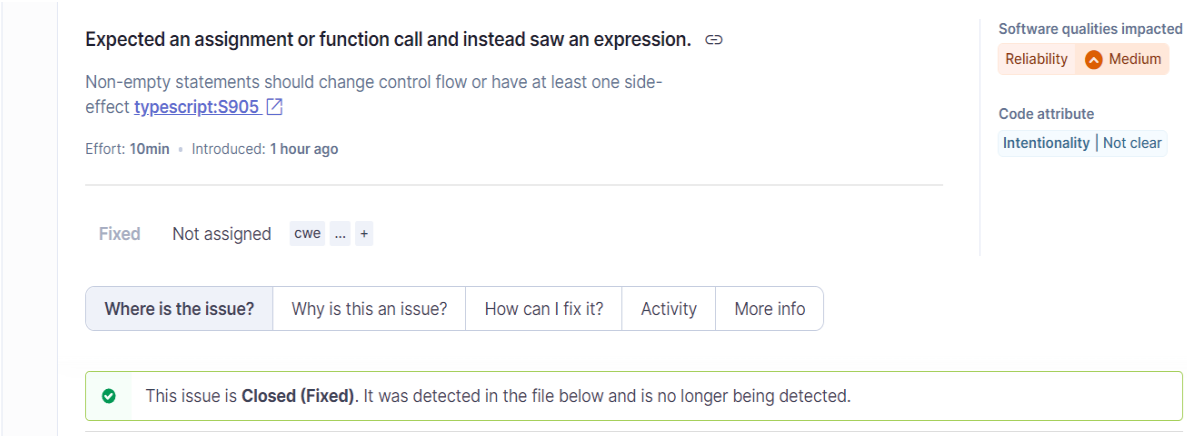


Slika 9: Prikaz problema nepravilne upotrebe comma operatora

Nepravilna upotreba zareznog operatora narušava čitljivost koda i može dovesti do neočekivanog ponašanja aplikacije. Ispravljanjem ovog problema, kroz pravilno definisanje callback funkcija unutar subscribe metode, obezbeđena je jasna struktura koda, povećana pouzdanost izvršavanja i uklonjeni su warning-i detektovani tokom statičke analize.

- **Izrazi bez efekta izvršavanja**

SonarQube je identifikovao više slučajeva u kojima su se u kodu pojavljivali izrazi koji ne predstavljaju ni dodelu vrednosti niti poziv funkcije. Ovakvi izrazi nemaju efekat na izvršavanje programa i često nastaju kao posledica pogrešne strukture asinhronog koda ili ostataka nakon refaktorisanja.



Slika 10: Prikaz problema izraza bez efekta izvršavanja

Prisustvo ovakvih izraza smanjuje jasnoću implementacije i može dovesti do pogrešne interpretacije logike aplikacije. Njihovim uklanjanjem ili pravilnim uključivanjem u funkcionalni tok izvršavanja, unapređena je pouzdanost koda i usklađenost sa preporučenim TypeScript praksama.

- **Neiskorišćeni importi u TypeScript fajlovima**

U okviru frontend projekta identifikovani su neiskorišćeni importi u više komponenti i dijaloga. Ovi importi nisu korišćeni u implementaciji klasa, ali su ostali prisutni u izvornom kodu.

Neiskorišćeni importi ne utiču direktno na rad aplikacije, ali narušavaju preglednost koda i povećavaju kognitivno opterećenje programera. Njihovim uklanjanjem postignuta je jasnija struktura fajlova i veća doslednost u organizaciji zavisnosti.

- **Neiskorišćeni i zakomentarisani delovi koda**

Tokom analize otkriveni su delovi zakomentarisanog koda u komponentama koji više nemaju funkcionalnu svrhu. Ovakvi delovi koda predstavljaju oblik tehničkog duga i mogu izazvati konfuziju prilikom daljeg razvoja aplikacije.

Uklanjanjem zakomentarisanog i neiskorišćenog koda unapređena je čitljivost komponenti, pojednostavljena struktura metoda i poboljšana održivost frontend sloja sistema.

- **Nedostatak pristupačnosti u interaktivnim HTML elementima**

SonarQube je detektovao probleme u HTML templejtima, gde interaktivni elementi, poput redova u tabelama, nisu posedovali odgovarajuće događaje za upravljanje putem tastature. Ovakav propust negativno utiče na pristupačnost aplikacije korisnicima koji ne koriste miša.

Dodavanjem odgovarajućih događaja za obradu pritiska tastera, obezbeđena je veća pristupačnost korisničkog interfejsa i usklađenost sa standardima inkluzivnog dizajna.

- **Neizmenjivi članovi klasa (readonly)**

U servisima i komponentama frontend aplikacije identifikovani su članovi klasa koji se nikada ne menjaju nakon inicijalizacije, ali nisu označeni kao readonly. SonarQube ovakvu praksu označava kao problem održivosti.

Označavanjem ovih članova kao readonly unapređena je jasnoća namene promenljivih, smanjena mogućnost neželjenih izmena i poboljšana stabilnost koda.

- **Neoptimalna inicijalizacija aplikacije**

U glavnom fajlu aplikacije detektovana je upotreba promise lanca za inicijalizaciju Angular aplikacije, umesto korišćenja savremenijeg top-level await pristupa.

Zamenom promise lanca sa top-level await konstrukcijom unapređena je čitljivost inicijalizacionog koda i usklađenost sa modernim ECMAScript standardima.

Opis rešenja

Za svaki detektovani problem urađena je korekcija u izvornom kôdu. Ovo poglavlje sadrži:
- Opis svakog tipa problema
- Originalni kod (pre korekcije)
- Novi kod (posle korekcije)
- Obrazloženje zašto je promena urađena

Cilj je da se kroz ove izmene postigne stanje u kojem SonarQube ne detektuje nikakve preostale probleme i da kvalitet koda zadovoljava sve relevantne standarde.

Maven projekat

Za svaki tip problema identifikovan tokom statičke analize koda pomoću SonarQube alata urađena je korekcija u izvornom kodu.

1. Neiskorišćeni importi

Opis problema:
Detektovani su neiskorišćeni importi u velikom broju klasa, uključujući REST kontrolere, servisni sloj, modele i integracione testove. Ovi importi ne utiču direktno na izvršavanje aplikacije, ali smanjuju preglednost koda i mogu dovesti do pogrešne interpretacije zavisnosti.

Primer originalnog koda:
```
import org.springframework.beans.factory.annotation.Autowired;
import rva.models.Banka;
```

Novi kod:
```
// Neiskorišćeni importi uklonjeni
```

Obrazloženje:
Uklanjanjem neiskorišćenih import izjava smanjuje se vizuelna složenost koda, povećava preglednost i profesionalni izgled, kao i olakšava refaktorisanje i buduće održavanje.

Primeri gde se javljao problem i kako je rešen:
BankaController - uklonjen import `org.springframework.beans.factory.annotation.Autowired`
FilijalaController - uklonjen import `rva.models.Usluga`
KorisnikUslugeController - uklonjeni importi `rva.models.Banka` i `rva.models.Filijala`
UslugaController - uklonjen import `rva.models.Banka`

2. Korišćenje generičkog wildcard tipa

Opis problema:
Generički wildcard tip (?) korišćen u povratnim vrednostima REST metoda (ResponseEntity<?>) smanjuje tip-sigurnost i eksplicitnost API-ja, otežavajući statičku analizu koda i održavanje.

Primer originalnog koda:
```
@GetMapping("/banka/id/{id}")
public ResponseEntity<?> getBankaById(@PathVariable int id) {
    Optional<Banka> banka = service.findById(id);
    if(banka.isPresent()) {
        return ResponseEntity.ok(banka.get());
    }
    return ResponseEntity.status(404).body("Resource with requested ID: "+ id + "
does not exist");
}
```

Novi kod:
```
@GetMapping("/banka/id/{id}")
public ResponseEntity<Banka> getBankaById(@PathVariable int id) {
    Optional<Banka> banka = service.findById(id);
    return banka.map(ResponseEntity::ok)
                .orElseGet(() -> ResponseEntity.status(404).build());
}
```

Obrazloženje:
Zamenom wildcard tipa sa konkretnim tipom Banka unapređena je tip-sigurnost i eksplicitnost metode, što olakšava održavanje, čini API jasnijim i omogućava bolju statičku analizu.

Primeri gde se javljao problem i kako je rešen:
BankaController, FilijalaController, KorisnikUslugeController, UslugaController - sve metode koje su vraćale ResponseEntity<?> sada vraćaju konkretne tipove (Banka, Filijala, KorisnikUsluge, Usluga).

3. Field injection umesto constructor injection

Opis problema:
U REST kontrolerima i servis implementacijama korišćen je field injection (@Autowired na polju). Ovakav pristup smanjuje preglednost zavisnosti, otežava unit testiranje i kontroliše životni ciklus objekata.

Primer originalnog koda:
```
@Autowired
private BankaService service;
```

Novi kod:
```
private final BankaService service;
public BankaController(BankaService service) {
    this.service = service;
}
```

Obrazloženje:
Constructor injection jasno definiše sve zavisnosti, poboljšava testabilnost i održivost koda, olakšava kontrolu životnog ciklusa objekata i smanjuje tehnički dug.

Primeri gde se javljao problem i kako je rešen:
BankaController, FilijalaController, KorisnikUslugeController, UslugaController - promenjeno na constructor injection.
Servis implementacije (BankaServiceImpl, FilijalaServiceImpl, KorisnikUslugeServiceImpl, UslugaServiceImpl) - sve field injection instance zamenjene constructor injection.

4. Dupliranje literala i dizajnerske nepravilnosti

Opis problema:
Identifikovana su višestruka ponavljanja istih string literala, npr. poruka `"Resource with requested ID ... does not exist"`, što otežava održavanje i krši princip jedinstvene odgovornosti.

Primer originalnog koda:
```
return ResponseEntity.status(404).body("Resource with requested ID: "+ id + " does not exist");
```

Novi kod:
```
private static final String RESOURCE_NOT_FOUND = "Resource with requested ID does not exist";
return ResponseEntity.status(404).body(RESOURCE_NOT_FOUND + ": " + id);
```

Obrazloženje:
Korišćenjem konstanti eliminiše se dupliranje, povećava konzistentnost poruka i olakšava održavanje, što smanjuje rizik od grešaka i tehničkog duga.

Primeri gde se javljao problem i kako je rešen:
BankaController, FilijalaController, KorisnikUslugeController, UslugaController - sve poruke koje obaveštavaju o nepostojanju resursa su centralizovane kroz konstante.

5. Neiskorišćeni i komentarisani delovi koda u testovima

Opis problema:
U integracionim testovima detektovani su blokovi zakomentarisnog ili neiskorišćenog koda koji smanjuju preglednost i mogu dovesti do konfuzije.

Primer originalnog koda:
```
// @Test
// public void testGetBanka() {
//
// }
import rva.models.Banka; // neiskorišćen import
```

Novi kod:
```
// Zakomentarisani i neiskorišćeni delovi uklonjeni
```

Obrazloženje:
Uklanjanjem neiskorišćenog i komentarisnog koda poboljšava se preglednost testova, olakšava razumevanje testnih scenarija i smanjuje tehnički dug.

Primeri gde se javljao problem i kako je rešen:
BankaControllerIntegrationTest, FilijalaControllerIntegrationTest, KorisnikUslugeControllerIntegrationTest, UslugaControllerIntegrationTest - uklonjeni komentarisani blokovi i neiskorišćeni importi.

6. Upotreba nesigurnog pseudonasumičnog generatora

Opis problema:
U klasi TestServer korišćen je `Math.random()` za generisanje nasumičnih brojeva. SonarQube označava ovakav pristup kao potencijalno nesiguran, posebno u kontekstu gde se vrednosti mogu koristiti u poslovnoj logici ili prenositi u kritične operacije. Math.random() koristi pseudonasumične algoritme koji nisu kriptografski sigurni, što može biti problem u nekim scenarijima.

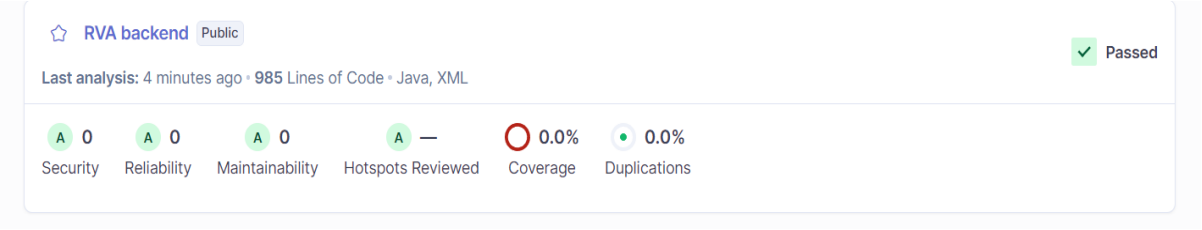
Originalni kod:
```
package rva;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class TestServer {
    @GetMapping("/zbir")
    public double zbirBrojeva() {
        double prvi = Math.random() * 10;
        double drugi = Math.random() * 10;
        return prvi + drugi;
    }
}
```

Novi kod:
```
package rva;
import java.security.SecureRandom;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class TestServer {
    private static final SecureRandom secureRandom = new SecureRandom();
    @GetMapping("/zbir")
    public double zbirBrojeva() {
        double prvi = secureRandom.nextDouble() * 10;
        double drugi = secureRandom.nextDouble() * 10;
        return prvi + drugi;
    }
}
```

Obrazloženje:
Korišćenjem SecureRandom umesto Math.random() poboljšava se sigurnost generisanih nasumičnih brojeva, čineći ih pogodnim za upotrebu u kontekstu gde je potrebna veća sigurnost i nepredvidivost. Ova promena takođe eliminiše SonarQube upozorenje i doprinosi standardima dobre prakse u Java backend razvoju.

Primeri gde se javljao problem i kako je rešen:
TestServer - svi nasumični brojevi sada se generišu pomoću SecureRandom
Metoda zbirBrojeva koristi privatnu statičku instancu SecureRandom za konzistentno i sigurno generisanje vrednosti

Ovo poglavlje sada pokriva sve tipove problema koje je SonarQube detektovao u celom backend Maven projektu, sa opisom, originalnim kodom, novim kodom i obrazloženjem zašto je korekcija urađena, uključujući i mesta gde se problem javljao i kako je rešen.
Nakon otklanjanja svih pomenutih problema, analiza projekta je prošla i rezultati su prikazani na slici ispod.



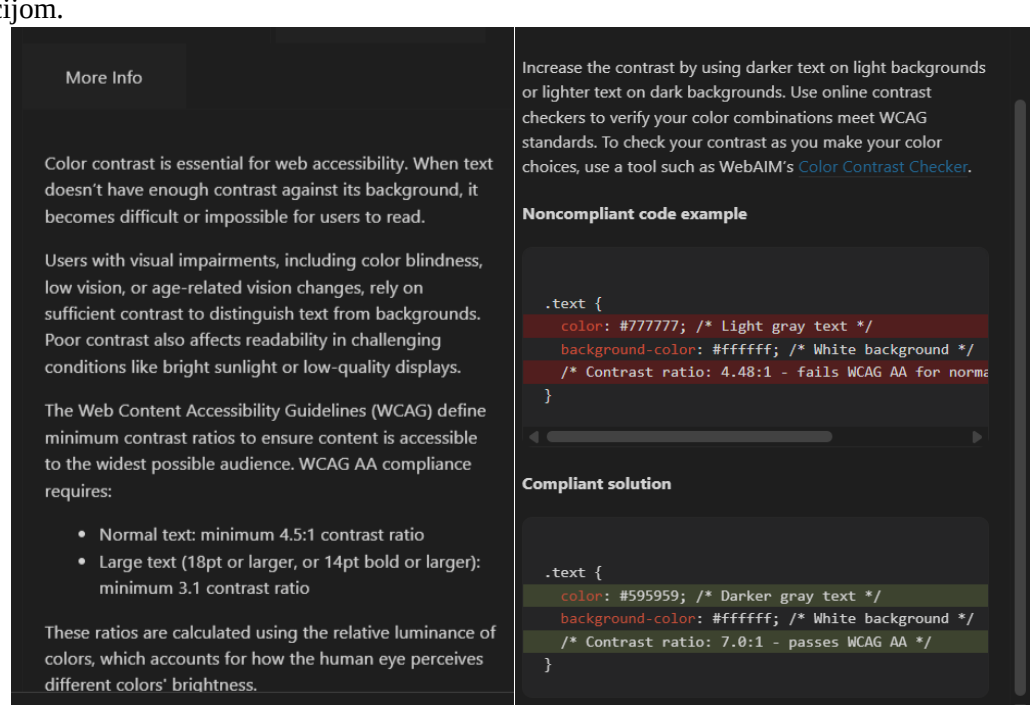
Slika 11: Rezultati analize Maven projekta

Angular projekat

Tokom statičke analize frontend Angular aplikacije primenom SonarQube Scanner alata, identifikovane su višestruke nepravilnosti koje se odnose na pristupačnost, pouzdanost, održivost i konzistentnost izvornog koda. U nastavku su detaljno opisani svi tipovi problema, uz konkretne primere iz aplikacije i obrazloženja izvršenih korekcija.

1. Problemi pristupačnosti (Accessibility) - nedovoljan kontrast teksta u odnosu na pozadinu

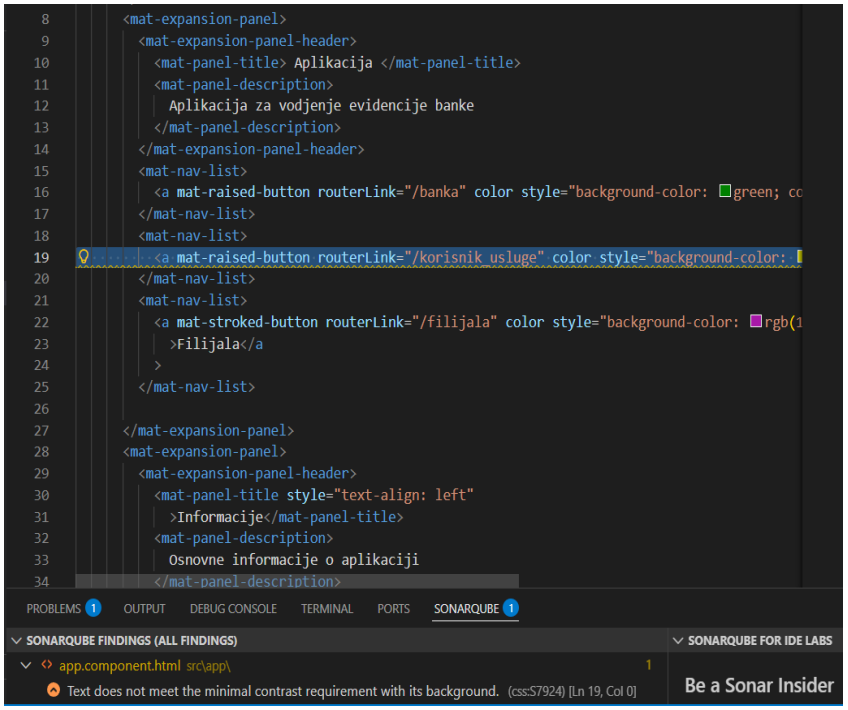
Opis problema
Primarni problem uočen u inicijalnom rešenju bio je nizak nivo kontrasta uzrokovan upotrebom svetložute pozadine i bele boje teksta. Ovakva kombinacija direktno narušava čitljivost digitalnog sadržaja, čineći ga gotovo neupotrebljivim za korisnike sa oštećenim vidom. Pored subjektivnog zamora očiju, ovakav propust predstavlja i objektivno odstupanje od međunarodno priznatih standarda pristupačnosti, što može dovesti do isključivanja značajnog broja korisnika iz interakcije sa aplikacijom.



Slika 13:Prikaz primera neadekvatnog i adekvatnog kontrasta boja u izvornom kodu (CSS) prema WCAG standardima

Jedan od ključnih aspekata inkluzivnog dizajna softverskih interfejsa jeste osiguravanje adekvatnog nivoa kontrasta između teksta i pozadine. Prema smernicama za pristupačnost veb sadržaja (WCAG 2.1), neophodno je postići određene koeficijente osvetljenosti kako bi informacije bile čitljive svim grupama korisnika, uključujući osobe sa oštećenjem vida, daltonizmom ili senilnom degeneracijom makule.

Primer originalnog koda je prikazan na slici ispod:



Slika 12: Nedovoljan kontrast teksta u odnosu na pozadinu

Korekcija je izvršena modifikacijom vrednosti boja u CSS datotekama, čime je postignut značajno viši nivo vizuelnog kontrasta. Rezultat ove promene nije samo poboljšana čitljivost i estetika interfejsa, već i podizanje opšteg kvaliteta softverskog rešenja. Implementacijom ovih promena uspešno su uklonjena SonarQube upozorenja vezana za pristupačnost, čime je potvrđeno da aplikacija sada ispunjava tehničke norme predviđene za savremene digitalne proizvode.

2. Problemi pouzdanosti i namere koda (Reliability & Intentionality)

2.1 Neispravna upotreba subscribe metode (comma operator)

Opis problema
U više komponenti SonarQube je detektovao neočekivanu upotrebu comma operatora prilikom definisanja subscribe metode.

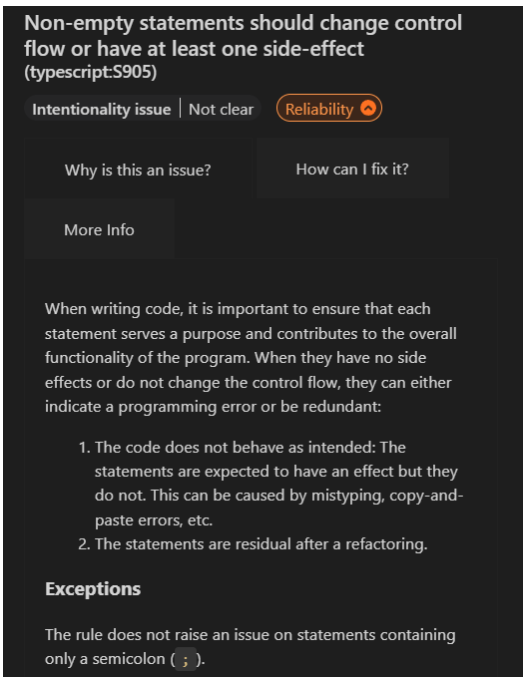
Originalni kod

```
22   public add() {
23     this.service.addBanka(this.data).subscribe(
24       (data) => {
25         this.snackBar.open(`Uspesno dodata banka sa nazivom: ` + (this.data.naziv), `U redu
26       }
27       ),
28       (error:Error) => {
29         console.log(error.name + ' ' + error.message);
30         this.snackBar.open(`Neuspesno dodavanje`, `Zatvori`, {duration:1000});
31       }
32     }
33
```

Slika 13: Primer neispravne upotrebe comma operatora

Zašto je ovo problem?

- Subscribe prihvata parametre unutar jedne funkcije
- Comma operator stvara izraz bez jasne namere
- SonarQube ovo tumači kao potencijalnu logičku grešku



Slika 14: Prikaz smernica za ispravljanje problema comma operatora

Ispravljeni kod je prikazan na slici ispod.

```
22  public add() {
23    this.service.addBanka(this.data).subscribe(
24      (data) => {
25        this.snackBar.open(
26          `Uspesno dodata banka sa nazivom: ${this.data.naziv}`,
27          `U redu`,
28          { duration: 2500 }
29        );
30      },
31      (error: Error) => {
32        console.log(error.name + ' ' + error.message);
33        this.snackBar.open(`Neuspesno dodavanje`, `Zatvori`, { duration: 1000 });
34      }
35    );
36  }
```

Slika 15: Rešenje problema vezanog za upotrebu comma operatora

Korekcija navedenog problema izvršena je kroz restrukturiranje argumentacije unutar subscribe metode, čime je eliminisana semantička dvosmislenost koju uvodi comma operator. Umesto korišćenja zapete za razdvajanje izraza unutar jednog callback-a (što je SonarQube detektovao kao logičku grešku jer takvi izrazi nemaju jasan efekat na tok programa), primenjena je standardna sintaksa sa blokom instrukcija unutar vitičastih zagrada {}.
Ovim pristupom postignuta je jasna separacija između logike za uspešnu obradu podataka i logike za tretiranje grešaka. Definisanje next i error povratnih funkcija kao zasebnih argumenata osigurava da svaki blok koda ima precizno definisanu ulogu i opseg delovanja. Krajnji rezultat je kod koji je u potpunosti usklađen sa najboljim praksama RxJS biblioteke i TypeScript standarda, čime je povećana robusnost aplikacije i eliminisana mogućnost nepredviđenog ponašanja tokom izvršavanja asinhronih operacija.

2.2 „Expected an assignment or function call and instead saw an expression“

Opis problema
Ova greška u TypeScript-u ukazuje na to da prevodilac (compiler) nailazi na izolovani izraz unutar koda koji ne vrši nikakvu promenu stanja (nema dodeljivanja vrednosti promenljivoj - assignment) niti pokreće izvršavanje neke akcije (nema poziva funkcije - function call). U konkretnom slučaju, uočena je definicija anonimne arrow funkcije (callback-a za grešku) koja stoji samostalno i koja je sintaksički i logički pozicionirana van opsega subscribe poziva. Kako se ovaj izolovani kod nikada ne može izvršiti, prevodilac ga interpretira kao suvišan i potencijalno neispravan izraz, što narušava strukturu metode, smanjuje čitljivost i direktno utiče na pouzdanost i robusnost aplikacije.

Originalan kod:

```
(error: Error) => {
  console.log(error.message);
}
(pozicionirano van subscribe poziva)
```

Rešenje identifikovanog problema postignuto je pravilnim ugnježđivanjem callback funkcije za rukovanje greškama (error callback) unutar konfiguracionog objekta subscribe metode. Premeštanjem definicije funkcije iz izolovanog položaja u predviđeni opseg, obezbeđeno je da se ona izvršava isključivo u slučaju da asinhroni tok rezultira greškom, u skladu sa najboljim praksama RxJS biblioteke i TypeScript standarda. Ovim korakom je otklonjena semantička dvosmislenost koda, eliminisana prevodilačka greška i postignuta puna sintaksička ispravnost.

3. Neiskorišćen i zakomentarisani kod

Opis problema:
Prisustvo zakomentarisanih segmenata koda (tzv. "dead code") unutar aktivne baze koda direktno doprinosi akumulaciji tehničkog duga. Ovakva praksa narušava kognitivni protok programera prilikom pregleda koda, otežavajući razlikovanje aktivne logike od napuštenih fragmenata. S obzirom na to da savremeni sistemi za kontrolu verzija (npr. Git) čuvaju potpunu istoriju svih izmena, zadržavanje zakomentarisanih linija unutar produkcionih datoteka je redundantno i metodološki neispravno.

Primer originalnog koda:
```
// console.log(data);
```

Primenjena je strategija potpunog uklanjanja svih zakomentarisanih blokova i neiskorišćenih instrukcija. Time je postignuta maksimalna čitljivost i "čistoća" koda, dok je istorijska evidencija izmena ostavljena na upravljanje sistemu za kontrolu verzija, čime je optimizovan proces održavanja softvera.

4. Neiskorišćeni importi

Opis problema:
Zadržavanje neiskorišćenih import izjava stvara lažnu sliku o zavisnostima modula, što može dovesti do pogrešnih zaključaka prilikom refaktorisanja ili analize arhitekture sistema. Pored toga, suvišni importi vizuelno opterećuju zaglavlja datoteka i otežavaju navigaciju kroz stvarne zavisnosti komponente, što dugoročno usporava proces razvoja i održavanja.

Primer originalnog koda:
```
import { Filijala } from 'src/app/models/filijala';
import { FilijalaService } from 'src/app/services/filijala.service';
```

Rešenje:
Izvršena je revizija svih modula i uklanjanje svih deklaracija koje se ne koriste u telu klase. Rezultat je precizno definisana struktura zavisnosti, što olakšava razumevanje povezanosti između različitih delova aplikacije i optimizuje proces kompajliranja.

5. Poboljšanje održivosti - readonly atributi

Opis problema:
Nedostatak eksplicitne definicije nepromenljivosti (immutability) za servise unutar konstruktora ostavlja prostor za nenamerne modifikacije referenci tokom životnog ciklusa komponente. Bez modifikatora pristupa koji garantuje nepromenljivost, stabilnost aplikacije može biti kompromitovana ljudskom greškom.

Primer originalnog koda
```
constructor(private service: BankaService) {}
```

Ispravljeni kod
```
constructor(private readonly service: BankaService) {}
```

Uveden je readonly atribut za sve zavisnosti injektovane putem konstruktora. Ova praksa jasno izražava arhitektonsku nameru da reference na servise moraju ostati konstantne nakon inicijalizacije.

6. Pristupačnost interaktivnih elemenata u tabelama

Opis problema:
Standardni redovi u tabelama (<mat-row>) koji reaguju isključivo na click događaj nisu dostupni korisnicima koji navigaciju vrše putem tastature ili pomoćnih tehnologija (screen readers). Nedostatak fokusabilnosti i podrške za događaje sa tastature predstavlja ozbiljan propust u pogledu digitalne inkluzije i kršenje WCAG standarda.

Originalan kod:
```
<mat-row
  *matRowDef="let row; columns: displayedColumns"
  (click)="selectRow(row)">
</mat-row>
```

Ispravljen kod:
```
<mat-row
  tabindex="0"
  (keydown)="selectRow(row)"
  (click)="selectRow(row)">
</mat-row>
```

Unapređena je interaktivnost tabela dodavanjem atributa tabindex="0", čime je omogućeno fokusiranje redova tasterom Tab. Dodatno, implementiran je keydown slušalac događaja koji omogućava aktivaciju akcije putem tastature, čime je postignuta potpuna usklađenost sa standardima pristupačnosti i omogućeno ravnopravno korišćenje aplikacije svim grupama korisnika.

7. Implementacija savremenih ECMAScript standarda (Top-level await)

Opis problema:
Tradicionalni pristup asinhronom podizanju aplikacije (bootstrapping) putem .catch() sintakse, iako funkcionalan, često vodi ka manje preglednom kodu, naročito u kompleksnim asinhronim tokovima. Sa uvođenjem ES2022 standarda, korišćenje top-level await postaje preporučena praksa za asinhronu kontrolu toka na nivou modula.

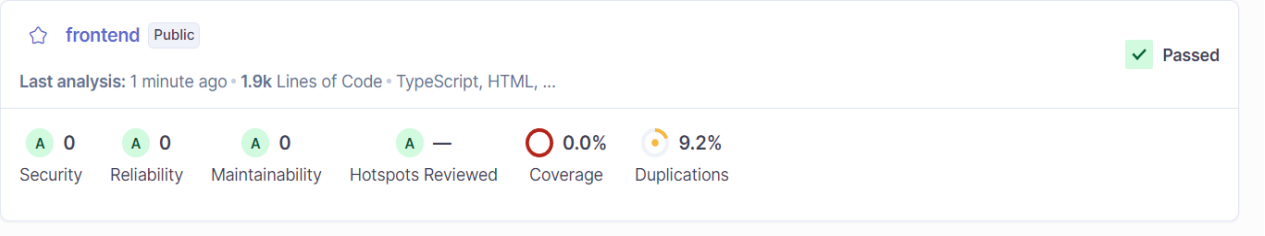
Originalni kod
```
platformBrowserDynamic()
  .bootstrapModule(AppModule)
  .catch(err => console.error(err));
```

Izmenjen kod:
```
try {
  await platformBrowserDynamic().bootstrapModule(AppModule);
} catch (err) {
  console.error(err);
}
```

Kod za pokretanje aplikacijskog modula refaktorisan je upotrebom await operatora na najvišem nivou. Ovim pristupom eliminisana je potreba za ulančavanjem povratnih poziva, kod je postao čitljiviji, a kontrola eventualnih grešaka tokom inicijalizacije aplikacije postala je intuitivnija i pouzdanija.

Za svaki detektovani problem izvršene su korekcije u izvornom kodu. Nakon primene izmena, kvalitet frontend aplikacije je značajno unapređen, a aplikacija je usklađena sa preporučenim industrijskim i akademskim standardima.

Na slici ispod su prikazani detalji analize Angular projekta.



Slika 16: Detalji analize Angular projekta pomoću SonarScanner alata

Konačna analiza projekta, prikazana na slici iznad, služi kao empirijska potvrda uspešnosti svih sprovedenih refaktorisanja. Postizanje statusa "Passed" na SonarQube platformi ukazuje na to da su kritični parametri softverskog kvaliteta - bezbednost (Security), pouzdanost (Reliability) i održivost (Maintainability) dovedeni na optimalan nivo. Nula detektovanih ranjivosti i grešaka u kodu direktno reflektuje primenu strožih pravila razvoja, dok niska stopa duplikacija garantuje lakše buduće održavanje sistema.

Sveukupno, proces identifikacije i otklanjanja anomalija u izvornom kodu nije rezultovao samo tehničkim poboljšanjem performansi aplikacije, već je osigurao i visok stepen inkluzivnosti kroz ispunjavanje standarda pristupačnosti. Ovakav pristup razvoju, koji kombinuje automatizovanu statičku analizu sa svesnim dizajnom usmerenim ka korisniku, postavlja temelj za dugovečnost softverskog proizvoda i usklađenost sa savremenim inženjerskim normama.

Literatura

1. Lenarduzzi, V., Saarimäki, N., & Taibi, D. (2019). *The Technical Debt Dataset*. arXiv. The Technical Debt Dataset
2. Esposito, M., Robredo, M., Arcelli Fontana, F., & Lenarduzzi, V. (2024). *On the correlation between Architectural Smells and Static Analysis Warnings*. https://arxiv.org/abs/2406.17354?
3. Yeboah, J., & Popoola, S. (2024). *Efficacy of static analysis tools for software defect detection on open-source projects*. https://doi.org/10.48550/arXiv.2405.12333
4. SonarSource (2025) *SonarQube Analysis Overview (Scanner Engine & Language Analyzers)*. SonarSource Documentation. https://www.sonarsource.com/resources/library/static-code-analysis-using-sonarqube/
5. SonarSource (2024) *Static Code Analysis Using SonarQube*. SonarSource Resource Guide. https://www.sonarsource.com/resources/library/static-code-analysis-using-sonarqube/
6. Golovyrin, L. (2023). *Analysis of tools for static security testing of applications*
7. Onyenweaku, I. R., Brown, M. S., Pelosi, M., & Shahine, M. H. (2021). *A SonarQube Static Analysis of the Spectral Workbench*. International Journal of Natural Science and Reviews.
8. T. Pol. (2021). *SonarQube rule violations that actually matter: An empirical investigation of SonarQube rule classifications*.
9. V. Lenarduzzi, N. Saarimäki, & D. Taibi (2019). *The Technical Debt Dataset*.
10. Valentina Lenarduzzi, Nyyti Saarimäki & Davide Taibi (2019). *Some SonarQube Issues have a Significant but Small Effect on Faults and Changes: A Large-scale Empirical Study*. https://arxiv.org/pdf/1908.11590
11. SonarSource. (2025). *Understanding measures and metrics*. SonarQube Server Documentation. https://docs.sonarsource.com/sonarqube-server/latest/user-guide/code-metrics/metrics-definition/
12. SonarSource. (2025). *Monitoring project metrics*. SonarQube Server Documentation. https://docs.sonarsource.com/sonarqube-server/user-guide/code-metrics/monitoring-project-metrics/
13. Quezada Sarmiento, D. G., et al. (2019). *SonarQube as a tool to identify software metrics and technical debt in the source code through static analysis*. ResearchGate PDF. ResearchGate